Homework 1: Math and JavaScript

CS 440 Computer Graphics
Habib University
Spring 2026

Due: 11:59pm on Friday, 30 January

1. Building Blocks: Vectors and Matrices

Points, vectors, and matrices form the foundation of computer graphics, and much of the remainder of this course relies on using them correctly. In this problem, you will implement constructor functions for vectors and matrices of varying dimensions. These functions will be used throughout the assignment (and in our future assignments) to represent geometric data in JavaScript.

Task 1: Write JavaScript functions named `vec2`, `vec3`, and `vec4` that construct 2, 3, and 4 dimensional vectors, respectively.

Task 2: Write JavaScript functions named `mat2`, `mat3`, and `mat4` that construct 2×2, 3×3, and 4×4 matrices in row-major order, respectively.

Each function should:

- Accept its components as separate arguments according to its dimension e.g. `vec2(1, 2)`, `mat2(1, 2, 3, 4)` ([1, 2] for first row, [3, 4] for second row of the matrix).
- Verify that the correct number of components has been provided e.g. `vec2` must receive exactly two components.
- Return a vector/matrix representation that allows component access using index notation e.g. `v[0]` for vectors, `m[0][0]` for matrices.

Note that you are not required to create a class for vectors and matrices, simply creating a vector/matrix function with appropriate arguments and return values is sufficient. You also do not need to display vectors/matrices anywhere on your HTML page.

Files: common.js

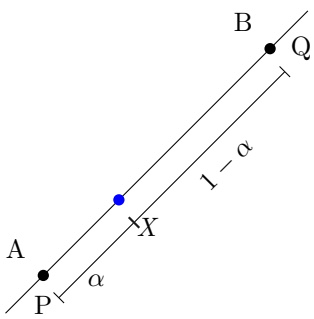
2. Taking it for a Spin

In this problem we will utilize the functions you created in Problem 1. Write a JavaScript script which asks the user for a dimension between 2 and 4 inclusive, and then asks the user to enter 2 vectors of that dimension. It then presents the user a menu with the following choices.

- Tell whether the vectors are equal
- Show the lengths of the vectors
- Show the normalized vectors
- Show the sum of the vectors
- Show the difference of the vectors
- Show the dot product of the vectors
- Show the cross product of the vectors
- Exit

The program should repeat until the user chooses to exit. Only use JavaScript and functions created in Problem 1. Do not use any higher level library. You may have to look up how to perform user I/O in JavaScript. Output typically uses alert() or console.log(), or you can choose to create text boxes using HTML and display them. Make sure to find out how to invoke your browser's console as that will often be your only means to debug your program.
Write your program in 2 files - operations.html and operations.js – where the program logic resides in operations.js and the HTML file includes operations.js and common.js (from Problem 1). Opening the HTML file in a browser should launch the script.
Files: operations.html, operations.js

3. Mapping and Linear Interpolation



In the figure on the left, X lies on the line segment PQ such that

$$|PX| : |XQ| = \alpha : (1 - \alpha)\ ,\ 0 \leq \alpha \leq 1,$$

which leads to

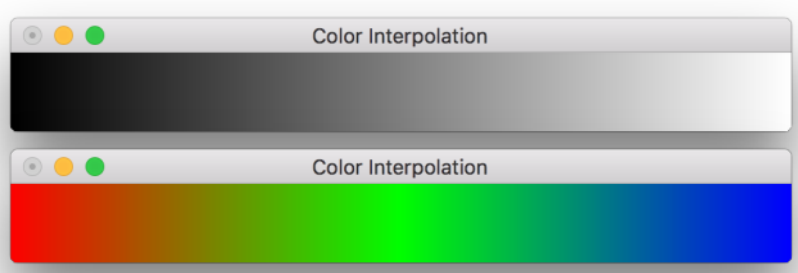
$$X = \alpha Q + (1 - \alpha)P.$$

That is, X is an *affine combination* of P and Q. In other words, X is obtained by linearly interpolating between P and Q.
Task 1: Write a function `lerp()` that takes three arguments: P, Q, and α. The function should return the point X, which is obtained by linearly interpolating between P and Q using the parameter α, computed component-wise.

Now, imagine a mapping from the range PQ to a new range AB. For example, P and Q may be vertices with colors A and B assigned to them respectively. We would be interested in finding out the color for X under this mapping. Generally, P, Q, and X belong to the same domain, and A and B are members of the same domain which may be different from the one that contains P, Q, and X.

Task 2: Write a function, `map_point()` that takes as argument, P, Q, A, B, and X. P, Q, and X are of the same type. A and B are of the same type. The function returns a value of the same type as A and B which is the mapping of X. Hint: You may need to use the `lerp()` function created in Task 1.

4. Color Interpolation



WebGL renders in a canvas on an HTML page. Individual pixels in the canvas have pixel coordinates which start at 0 on the left and increase rightward. Pixel coordinates are always whole numbers. WebGL imposes a coordinate system on the canvas in which the origin is at the middle of the canvas, the left and right extremes of the canvas have x values of -1 and 1 respectively, and the bottom and top extremes of the canvas have y values of -1 and 1 respectively.
Imagine interpolating colors as shown above using WebGL. The width of the canvas is equal to W pixels. In the figure on the top, the left extreme of the canvas corresponds to black `vec3(0,0,0)` and

the rightmost to white `vec3(1,1,1)`. The pixels between the horizontal extremes are different shades of gray.
In the figure on the bottom, the left extreme of the canvas corresponds to red `vec3(1,0,0)`, the center to green `vec3(0,1,0)`, and the right extreme to blue `vec3(0,0,1)`. The pixels between the left and the center have a color value interpolated between red and green. The pixels between the center and the right have a color value interpolated between green and blue.

Task: Write a script that inputs W, the width of the canvas, and a valid pixel coordinate, i.e. $\in [0, W)$, and uses your function from Problem 3 to output

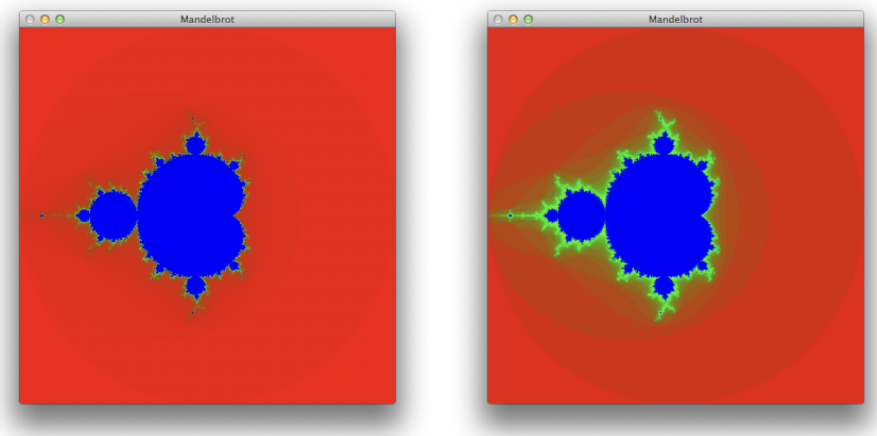
(a) the corresponding WebGL coordinate (assume y to be 0),

(b) the value of the corresponding shade of gray, and

(c) the value of the corresponding color.

Files: interpolate.html, interpolate.js

5. The Mandelbrot Set



Imagine visualizing the Mandelbrot set as shown above. The set consists of complex numbers lying on the circle of radius 2 centered at the origin in the complex plane. The visualizations above show the relevant portion of the complex plane; the top left corner corresponds to $-2 + 2i$ and the bottom right to $2 - 2i$. Each pixel in the visualization corresponds to a complex number. The colors assigned to the pixels in the visualizations are based on certain properties of the corresponding complex numbers, but they can be ignored for this problem. The canvas in both cases has the same width and height, L. As always, WebGL imposes its own coordinate system on the canvas as described in Problem 4.
Task: Write a script that takes as input L and a pixel position defined by its row and column numbers. In the context of the canvas, the origin is at the top left, the column number increases from left to right, and the row number increases from top to bottom. Your script should then use your function from Problem 3 to output

(a) the complex number corresponding to the input pixel, and

(b) the corresponding WebGL coordinate.

Files: mandelbrot.html, mandelbrot.js

Note that for Problems 4 and 5, you **do not** need to create the Color Interpolation or Mandelbrot visualizations, only the necessary functions required for their mathematical calculations. We will be visualizing them in WebGL as part of Assignment 2.

Submission Instructions

You will turn in your code via Canvas. Please submit a single zip file containing all the required files. The zip file must be named using your HU ID. For example, if my ID is aa04049, I would name my zip file as "04049.zip". Please **do not** include your name or initials anywhere in your submission. Also, do not create a .rar or .tar file – please create a .zip file.
Make sure to follow the specified file naming convention. Also make sure that you create separate files for each problem as specified. If you do not follow the submission instructions it is possible that you will get a small point deduction.